

데이터 분석 대시보드

(Streamlit)

Streamlit

- 랭체인 기반의 챗봇 기능을 손쉽게 웹 서비스 형태로 배포하고 시각화할 수 있게 해주는 파이썬 기반의 라이브러리
- 데이터 및 머신러닝 애플리케이션을 위한 파이썬 프레임워크
- 파이썬으로 웹 애플리케이션을 만들 수 있는 도구
- 대시보드 형태의 데이터 시각화 애플리케이션이나 챗GPT와 같은 채팅 앱을 빠르고 쉽게 개발할 수 있게 해줌
- Streamlit의 가장 큰 매력은 낮은 개발 허들 덕분에, 아이디어를 현실화하는 개념 증명 단계의 프로토타입 개발에 최적화되어 있다는 점
- 복잡한 설정이나 어려운 개념을 배울 필요 없이, 파이썬으로 데이터를 분석하는 것처럼 자연스럽게 웹 애플리케이션을 만들 수 있는 단순함에 있음

Streamlit

● 개발 환경 설정

- streamlit 애플리케이션을 로컬 환경에서 실행하기 위해서는 3.9x 이상 버전의 파이썬이 필요

1) 가상 환경 생성

```
(.venv)  
$ python -m venv .venv
```

2) 가상 환경 활성화(.venv 문구가 터미널에 추가되었는지 반드시 확인)

```
(.venv)  
$ source .venv/scripts/activate
```

3) 필요 패키지 설치

```
(.venv)  
$ pip install streamlit
```

Streamlit

● 개발 환경 설정

- streamlit 애플리케이션을 로컬 환경에서 실행하기 위해서는 3.9x 이상 버전의 파이썬이 필요

4) 주피터 노트북에서 첫번째 Streamlit 애플리케이션 만들기

```
%%writefile my_first_app.py
import streamlit as st

st.title('안녕하세요 Streamlit!')
st.write('이것은 나의 첫 번째 Streamlit 애플리케이션입니다.')
```

- 이 코드는 my_first_app.py라는 파일을 만들고 그 안에 Streamlit 코드를 작성한다.
- %%writefile은 주피터 노트북에서 파일을 만들 때 사용하는 특별한 명령어이다.

5) 실행하기

```
(.venv)
$ streamlit run my_first_app.py
```

Streamlit

- 기본 UI 컴포넌트

- 텍스트 출력 컴포넌트
 - 제목과 헤더 만들기

```
%%writefile text_componets.py
import streamlit as st

# 제목과 헤더 만들기
st.title('이것은 가장 큰 제목입니다')
st.header('이것은 큰 헤더입니다')
st.subheader('이것은 작은 헤더입니다')
```

- st.title()
 - title 메서드는 텍스트를 **제목으로 지정**하여 출력하는 데 사용하는 메서드
 - 화면에 출력되는 문구의 스타일은 사용하는 브라우저(크롬, 엣지, 사파리 등)에 따라 다르게 나타날 수 있음
- st.header() / st.subheader()는 내용을 구분할 때 사용
 - 특정 영역의 주요 정보를 나타내거나, 추가적인 정보를 제공하고자 할 때 사용
 - 주식 분석에서 주가 차트, 거래량 같은 섹션을 나눌 때 유용

Streamlit

- 기본 UI 컴포넌트

- 텍스트 출력 컴포넌트

- 일반 텍스트 표시하기: `st.text()`

```
# 일반 텍스트 표시하기
```

```
st.text('이것은 일반적인 텍스트입니다')
```

```
st.text('여러 줄로 텍스트를 작성할 수도 있습니다')
```

- 제목 외에도 설명이나 안내문을 보여줄 때는 `st.text()`를 사용
 - 입력한 그대로 화면에 표시
 - 글자 크기나 스타일을 바꾸지 않고 단순히 텍스트를 보여줌

Streamlit

- 기본 UI 컴포넌트

- 텍스트 출력 컴포넌트

- 마크다운으로 꾸미기: st.markdown()

- # 마크다운으로 꾸미기

- st.markdown('**이것은 굵은 글씨입니다**')

- st.markdown('*이것은 기울어진 글씨입니다*')

- st.markdown('이것은 `코드`처럼 보이는 글씨입니다')

- 입력한 문자열에 마크다운 문법을 적용해 출력하는 메서드
 - 간단한 기호로 텍스트를 꾸밀 수 있는 방법
 - 글자를 굵게 출력할 수 있는 마크다운 문법(** ***) 으로 “world!”를 감싸면, 텍스트가 굵게 출력되는 것 확인: st.write()와 비교
 - 별표 하나로 감싸면 기울어진 글씨
 - 백틱(`)으로 감싸면 코드처럼 보이는 글씨

Streamlit

- 기본 UI 컴포넌트

- 텍스트 출력 컴포넌트

- 만능 출력 함수: st.write()

```
# 만능 출력 함수
st.write('안녕하세요!')
st.write(123)
st.write([1, 2, 3, 4, 5])
```

- 텍스트를 화면에 출력하는 가장 기본적인 함수
 - 텍스트뿐만 아니라 숫자, 표, Matplotlib 등 다양한 데이터를 렌더링하는 기능을 제공
 - 다만, 어디까지나 단순한 형태로 렌더링하는 기능을 제공할 뿐임

Streamlit

- 기본 UI 컴포넌트

- 입력 컴포넌트

- 선택 상자 만들기 : st.selectbox()

```
%%writefile input_components.py
import streamlit as st

# 선택 상자 만들기 - 좋아하는 과일 선택
fruit = st.selectbox(
    '좋아하는 과일을 선택하세요',
    ['사과', '바나나', '오렌지', '포도']
)
st.write(f'당신이 선택한 과일은 {fruit}입니다')
```

- 사용자가 여러 선택지 중에서 하나를 고를 수 있게 해주는 것
 - 드롭다운 메뉴를 만들
 - 첫 번째 인자는 사용자에게 보여줄 설명
 - 두 번째 인자는 선택할 수 있는 항목들의 목록

Streamlit

- 기본 UI 컴포넌트

- 입력 컴포넌트

- 텍스트 입력 받기: st.text_input()

```
# 텍스트 입력받기
name = st.text_input('이름을 입력하세요')
age = st.number_input('나이를 입력하세요', min_value=0, max_value=120)

if name and age:
    st.write(f'{name}님은 {age}살입니다')
```

- 사용자로부터 텍스트를 입력 받을 수 있는 기능
 - text_input 메서드의 첫 번째 인자로 입력창 위에 표시될 레이블 지정
 - st.number_input()은 숫자를 입력

Streamlit

- 기본 UI 컴포넌트

- 입력 컴포넌트

- 슬라이더로 값 조정하기: `st.slider()`

```
# 슬라이더로 값 조정하기
temperature = st.slider('온도를 선택하세요', 0, 40, 25)
st.write(f'선택한 온도는 {temperature}도입니다')
```

- 마우스로 끌어서 값을 조정할 수 있는 도구
 - 숫자 범위를 쉽게 선택할 때 편리
 - 첫 번째 숫자는 최솟값, 두 번째 숫자는 최댓값, 세 번째 숫자는 기본값
 - 사용자는 마우스로 슬라이더를 움직여서 원하는 값을 선택할 수 있음

Streamlit

- 기본 UI 컴포넌트

- 입력 컴포넌트

- 라디오 버튼과 체크박스

- st.radio() : 여러 선택지 중에서 하나만 고를 때 사용

```
# 라디오 버튼
color = st.radio(
    '좋아하는 색깔을 선택하세요',
    ['빨강', '파랑', '초록']
)
```

- st.checkbox() : 예 또는 아니오처럼 간단한 선택을 할 때 사용

```
# 체크박스
agree = st.checkbox('이용약관에 동의합니다')

if agree:
    st.write('동의해주셔서 감사합니다!')
```

Streamlit

- 기본 UI 컴포넌트

- 입력 컴포넌트

- 여러 개 선택하기: st.multiselect()

```
# 여러 개 선택하기
hobbies = st.multiselect(
    '취미를 선택하세요 (여러 개 선택 가능)',
    ['독서', '영화감상', '운동', '여행', '음악감상']
)

if hobbies:
    st.write('선택한 취미:', hobbies)
```

- 사용자는 여러 개의 취미를 동시에 선택할 수 있고, 선택한 항목들이 목록으로 저장됨

Streamlit

- 기본 UI 컴포넌트

- 입력 컴포넌트

- 날짜와 시간 입력: `st.date_input()` / `st.time_input()`

```
# 날짜와 시간 입력
from datetime import datetime

today = st.date_input('날짜를 선택하세요')
current_time = st.time_input('시간을 선택하세요')

st.write(f'선택한 날짜: {today}')
st.write(f'선택한 시간: {current_time}')
```

- 사용자는 여러 개의 취미를 동시에 선택할 수 있고, 선택한 항목들이 목록으로 저장됨

Streamlit

- 기본 UI 컴포넌트

- 미디어 컴포넌트

- 이미지 표시하기: st.image()

```
%%writefile media_components.py

import streamlit as st

# 인터넷 이미지 표시
st.image('https://via.placeholder.com/300x200', caption='예시 이미지')

# 로컬 이미지 파일 표시 (파일이 있는 경우)
st.image('my_image.jpg', caption='내 이미지')
```

- 이미지의 주소나 파일 경로를 입력하면 웹페이지에 이미지가 나타남
 - caption을 사용하면 이미지 아래에 설명을 추가할 수 있음

Streamlit

● 페이지 설정과 레이아웃

• 페이지 설정

```
%%writefile page_config.py

import streamlit as st

st.set_page_config(
    page_title="학생 성적 관리",
    page_icon="📖",
    layout="wide"
)

st.title('페이지 설정이 완료된 애플리케이션')
st.write('이제 더 넓은 화면에서 볼 수 있습니다!')
```

- `st.set_page_config()`는 반드시 다른 Streamlit 코드보다 먼저 작성해야 함
 - 이 함수는 페이지의 전체적인 모습 결정
- `page_title`은 브라우저 탭에 표시되는 제목
 - 사용자가 여러 탭을 열어 놓았을 때 어떤 페이지인지 구분할 수 있게 도와 줌
- `page_icon`은 브라우저 탭에 표시되는 작은 아이콘
- `layout` 설정은 페이지가 화면을 어떻게 사용할지 결정
 - `centered` : 내용을 화면 가운데서 모아서 보여줌
 - `wide` : 화면 전체를 사용. 차트나 표를 크게 보여줘야 하는 데이터 분석 애플리케이션에서 유용

Streamlit

- 페이지 설정과 레이아웃

- 레이아웃 컴포넌트
 - 열로 나누어 배치하기

```
%%writefile columns_layout.py

import streamlit as st

st.title('학생 정보 관리 시스템')

# 4개의 열로 나누기
col1, col2, col3, col4 = st.columns(4)

with col1:
    st.metric("전체 학생 수", "245명")

with col2:
    st.metric("평균 점수", "82.5점")

with col3:
    st.metric("출석률", "94.2%")

with col4:
    st.metric("과제 제출률", "87.8%")
```

Streamlit

- 페이지 설정과 레이아웃

- 레이아웃 컴포넌트
 - 열의 크기 조정하기

```
# 첫 번째 열을 다른 열보다 2배 크게 만들기
col1, col2, col3 = st.columns([2, 1, 1])

with col1:
    st.write('메인 콘텐츠 영역 - 이 열이 가장 넓습니다')

with col2:
    st.write('통계 정보')

with col3:
    st.write('빠른 메뉴')
```

Streamlit

- 페이지 설정과 레이아웃

- 레이아웃 컴포넌트
 - 컨테이너로 묶기

```
with st.container():  
    st.subheader('이번 달 도서관 현황')  
    st.write('이 영역에는 도서관의 주요 통계가 표시됩니다')  
    st.metric("대출 도서 수", "1,245권")  
    st.metric("신규 회원", "+23명")
```

- 관련된 내용들을 하나로 묶어서 보여주고 싶을 때 사용
- 컨테이너는 내용을 논리적으로 구분해주고, 나중에 스타일을 적용할 때도 유용

Streamlit

- 페이지 설정과 레이아웃

- 레이아웃 컴포넌트
 - 확장 가능한 섹션

```
with st.expander("상세 통계 정보"):
    st.write("여기에는 자세한 분석 결과가 들어갑니다")
    st.write("평소에는 숨겨져 있다가 필요할 때만 펼쳐볼 수 있습니다")
```

- 내용이 많을 때는 접었다 펼 수 있는 섹션을 만들 수 있음
- 사용자가 제목을 클릭하면 내용이 펼쳐지고, 다시 클릭하면 접힘
- 선택적으로 보여줄 정보나 상세한 설명을 넣을 때 좋음

Streamlit

- 페이지 설정과 레이아웃

- 정보 표시 컴포넌트
 - 지표 카드 만들기: st.metric()

```
%%writefile metrics_demo.py
import streamlit as st

st.title('카페 매출 대시보드')

## 지표 카드 만들기
# 기본 지표
st.metric("오늘 매출", "450,000원")

# 변화량과 함께 표시
st.metric(
    label="일일 방문객",
    value="127명",
    delta="+23명"
)

# 음수 변화량 (빨간색으로 표시됨)
st.metric(
    label="재고 수량",
    value="85개",
    delta="-15개"
)
```

- 숫자와 변화량을 예쁘게 보여주는 카드를 만들. 다양한 지표를 표시할 때 매우 유용. label은 지표의 이름이고, value는 현재 값. delta는 변화량

Streamlit

- 페이지 설정과 레이아웃

- 정보 표시 컴포넌트

- 상태 메시지 표시하기

```
# 정보 메시지 (파란색)
st.info('시스템 점검이 예정되어 있습니다')

# 성공 메시지 (초록색)
st.success('데이터 백업이 완료되었습니다!')

# 경고 메시지 (노란색)
st.warning('일부 기능이 제한될 수 있습니다')

# 오류 메시지 (빨간색)
st.error('서버 연결에 실패했습니다')
```

- 사용자에게 중요한 정보를 전달할 때는 색깔이 있는 메시지 박스를 사용
 - 각각 다른 색깔과 아이콘으로 표시되어 사용자가 메시지의 중요도를 쉽게 파악

Streamlit

- 페이지 설정과 레이아웃

- 정보 표시 컴포넌트
 - 로딩 표시하기

```
# 로딩 표시하기
import time

with st.spinner('학생 데이터를 처리하는 중...'):
    # 실제로는 데이터를 불러오는 코드가 들어감
    time.sleep(3) # 3초 대기 (예시)

st.success('처리 완료!')
```

- 데이터를 불러오거나 계산하는 동안 사용자에게 진행 상황을 보여줄 때 유용

Streamlit

- 페이지 설정과 레이아웃

- 정보 표시 컴포넌트

- 빈 공간 관리하기

```
# 빈 공간 만들기
placeholder = st.empty()

# 나중에 내용 채우기
import time
time.sleep(2)
placeholder.text('검색 결과가 나타났습니다!')

# 내용 교체하기
time.sleep(2)
placeholder.success('검색이 완료되었습니다!')
```

- st.empty()로 만든 공간에는 나중에 어떤 내용이든 넣을 수 있고, 내용을 바꿀 수도 있음

Streamlit

- 데이터 표시와 시각화

- 데이터프레임과 테이블 표시
 - 데이터를 표로 보여주기

```
%%writefile dataframe_demo.py
import streamlit as st
import pandas as pd

# 학생 성적 데이터 만들기
data = {
    '이름': ['김철수', '이영희', '박민수', '최지연'],
    '수학': [85, 92, 78, 96],
    '영어': [88, 85, 90, 93],
    '과학': [90, 88, 85, 89]
}

df = pd.DataFrame(data)

st.title('학생 성적 데이터 표시하기')
st.dataframe(df)
```

Streamlit

- 데이터 표시와 시각화

- 데이터프레임과 테이블 표시

- 표 스타일링하기

```
# 온라인 쇼핑몰 판매 데이터
sales_data = {
    '상품명': ['노트북', '마우스', '키보드', '모니터', '헤드셋'],
    '가격': [1200000, 25000, 80000, 350000, 150000],
    '판매량': [15, 120, 85, 30, 45],
    '평점': [4.5, 4.2, 4.7, 4.1, 4.6]
}

df = pd.DataFrame(sales_data)

# 평점에 따라 색깔 적용
def color_rating(val):
    if val >= 4.5:
        color = 'green'
    elif val >= 4.0:
        color = 'orange'
    else:
        color = 'red'
    return f'color: {color}'

# 스타일 적용해서 표시
styled_df = df.style.format({
    '가격': '{:,}원',
    '판매량': '{:,}개',
    '평점': '{:.1f}점'
}).map(color_rating, subset=['평점'])

st.dataframe(styled_df)
```

Streamlit

- 데이터 표시와 시각화

- 데이터프레임과 테이블 표시

- 정적 테이블 만들기

```
# 날씨 정보 표시용 정적 테이블
weather_data = {
    '항목': ['최고 기온', '최저 기온', '습도', '강수 확률'],
    '값': ['28°C', '18°C', '65%', '30%']
}

weather_df = pd.DataFrame(weather_data)
st.table(weather_df)
```

- st.table()은 고정된 크기의 단순한 표를 만듦
 - 데이터가 많지 않고 모든 내용을 한 번에 보여주고 싶을 때 적합

Streamlit

- 데이터 표시와 시각화

- 기본 차트 컴포넌트

- 선 그래프 추세 보기

```
%%writefile basic_charts.py
import streamlit as st
import pandas as pd
import numpy as np
from datetime import datetime, timedelta

# 웹사이트 방문자 수 데이터 만들기
dates = [datetime.now() - timedelta(days=x) for x in range(30, 0, -1)]
visitors = np.random.randint(100, 500, 30)

visitor_df = pd.DataFrame({
    'Date': dates,
    'Visitors': visitors
})

st.title('기본 차트 예시')
st.subheader('웹사이트 일일 방문자 수')
st.line_chart(visitor_df.set_index('Date')['Visitors'])
```

- 시간에 따른 변화를 가장 직관적으로 보여주는 방법. 웹사이트 트래픽처럼 시간 순서가 중요한 데이터에 적합

Streamlit

- 데이터 표시와 시각화

- 기본 차트 컴포넌트
 - 막대 그래프 비교하기

```
# 도시별 인구 비교
population_data = {
    '서울': 9720000,
    '부산': 3390000,
    '인천': 2950000,
    '대구': 2410000,
    '대전': 1490000
}

population_df = pd.DataFrame(list(population_data.items()), columns=['도시', '인구수'])
st.subheader('주요 도시 인구 비교')
st.bar_chart(population_df.set_index('도시')['인구수'])
```

- 여러 카테고리의 값을 비교할 때 좋음
- 막대 높이로 값을 크기를 직관적으로 비교할 수 있음
- 도시별 인구수처럼 카테고리 간 비교에 효과적

Streamlit

- 데이터 표시와 시각화

- 기본 차트 컴포넌트

- 영역 차트로 누적 보기

```
# 월별 매출 구성 비교
monthly_data = pd.DataFrame({
    'Date': dates,
    '온라인_매출': np.random.randint(50, 150, 30),
    '오프라인_매출': np.random.randint(80, 200, 30),
    '모바일_매출': np.random.randint(30, 100, 30)
})

st.subheader('채널별 매출 구성 변화')
st.area_chart(monthly_data.set_index('Date'))
```

- 시간에 따른 여러 요소의 변화를 함께 보여줄 때 사용
 - 각 영역의 두께로 해당 채널의 매출 비중을 확인할 수 있고, 전체적인 추세도 파악할 수 있음

Streamlit

- 데이터 표시와 시각화

- plotly 차트 통합
- 인터랙티브 차트의 장점

```
%%writefile plotly_integration.py

import streamlit as st
import plotly.graph_objects as go
import pandas as pd
import numpy as np
from datetime import datetime, timedelta

st.title('Plotly 차트 통합')

# 제품 판매량 데이터 생성
dates = [datetime.now() - timedelta(days=x) for x in range(100, 0, -1)]
np.random.seed(42)
sales = np.random.randint(50, 200, 100)

# Plotly 선 그래프
fig = go.Figure()
fig.add_trace(go.Scatter(
    x=dates,
    y=sales,
    mode='lines',
    name='일일 판매량',
    line=dict(color='blue', width=2)
))

fig.update_layout(
    title='인터랙티브 판매량 차트',
    xaxis_title='날짜',
    yaxis_title='판매량 (개)',
    height=400
)

st.plotly_chart(fig, use_container_width=True)
```

Streamlit

- 데이터 표시와 시각화

- plotly 차트 통합
 - 산점도 차트 만들기

```
# 광고비와 매출의 관계
np.random.seed(42)
ad_spend = np.random.randint(10, 100, 50)
revenue = ad_spend * 2.5 + np.random.normal(0, 20, 50)

scatter_fig = go.Figure()
scatter_fig.add_trace(go.Scatter(
    x=ad_spend,
    y=revenue,
    mode='markers',
    name='데이터 포인트',
    marker=dict(
        size=8,
        color='lightblue',
        line=dict(width=1, color='navy')
    )
))

scatter_fig.update_layout(
    title='광고비 vs 매출 관계',
    xaxis_title='광고비 (만원)',
    yaxis_title='매출 (만원)',
    height=400
)

st.subheader('산점도 차트')
st.plotly_chart(scatter_fig, use_container_width=True)
```


Streamlit

- 데이터 표시와 시각화

- plotly 차트 통합
 - 파이 차트로 비율 보기

```
# 설문조사 결과
survey_data = {
    '매우 만족': 25,
    '만족': 40,
    '보통': 20,
    '불만족': 10,
    '매우 불만족': 5
}

pie_fig = go.Figure(data=go.Pie(
    labels=list(survey_data.keys()),
    values=list(survey_data.values()),
    hole=0.3
))

pie_fig.update_layout(
    title='고객 만족도 설문 결과',
    height=400
)

st.subheader('만족도 분포')
st.plotly_chart(pie_fig, use_container_width=True)
```

Streamlit

- 데이터 표시와 시각화

- plotly 차트 통합
 - 히스토그램으로 비율 보기

```
# 학생들의 키 분포
np.random.seed(42)
heights = np.random.normal(170, 10, 200)

hist_fig = go.Figure()
hist_fig.add_trace(go.Histogram(
    x=heights,
    nbinsx=20,
    name='키 분포',
    marker_color='lightgreen'
))

hist_fig.update_layout(
    title='학생 키 분포',
    xaxis_title='키 (cm)',
    yaxis_title='학생 수',
    height=400
)

st.subheader('키 분포 히스토그램')
st.plotly_chart(hist_fig, use_container_width=True)
```